

# ELI — Runtime Planning World

ELI v2.1.68

August 2026

## Contents

|                                                                             |          |
|-----------------------------------------------------------------------------|----------|
| <b>ELI Runtime Surfaces, Planning, World, Tools &amp; Plugins</b>           | <b>1</b> |
| Runtime surfaces (eli/runtime/, the response/introspection group) . . . . . | 1        |
| Planning / proactive (eli/planning/, 3.9k LOC) . . . . .                    | 2        |
| World / autonomy model (eli/world/, 1.5k LOC) . . . . .                     | 2        |
| Tools (eli/tools/, 8.9k LOC) . . . . .                                      | 2        |
| Plugins (eli/plugins/, 1.9k LOC) . . . . .                                  | 2        |
| Honest assessment . . . . .                                                 | 3        |

## ELI Runtime Surfaces, Planning, World, Tools & Plugins

The remaining subsystems: the runtime/ response/introspection surfaces, the proactive planning layer, the “world”/autonomy model, and the tools + plugin system.

### Runtime surfaces (eli/runtime/, the response/introspection group)

Beyond the grounding/evidence core (grounding\_and\_evidence.md), runtime/ holds a large family of modules that render user-visible answers and introspect the live system:

- **Introspection:** live\_introspection.py (runtime\_snapshot, last\_trace, stored\_user\_name, mine\_user\_fact\_candidates, build\_report, agents\_for\_action), deterministic\_introspection.py (classify\_diagnostic\_action → gather\_evidence → format\_evidence\_block → maybe\_handle — deterministic diagnostic answers).
- **Response assembly:** final\_response\_assembly.py, final\_response\_provider.py, response\_contracts.py, response\_packets.py, response\_policy.py, user\_visible\_response\_surface.py.
- **Personal memory surfaces:** personal\_memory\_surface.py, personal\_memory\_clean\_response.py, personal\_memory\_deep\_response.py, profile\_extractor.py.
- **Status:** frontier\_status.py, reasoning\_status.py, truth\_report.py, reflection.py.
- **Operator feed:** operator\_feed.py, operator\_feed\_normalized.py, operator\_state.py.

**Over-fragmentation flag (the headline weakness for this group):** there are ~15 closely-related modules here doing “render a grounded user-visible answer / introspect runtime” with overlapping responsibilities (three personal\_memory\_\* renderers; final\_response\_assembly and final\_response\_provider; operator\_feed and operator\_feed\_normalized). This is the clearest “added beside, not folded in” cluster in the codebase and the prime consolidation target — it should collapse to a handful of well-named modules with clear ownership of “who produces the final string.”

## Planning / proactive (eli/planning/, 3.9k LOC)

Background cognition + goal/queue machinery: - proactive\_daemon.py (1.0k) — the **self-improvement / proactive** loop. Uses a two-DB split: reads user-facing memory/conversation from the **user DB**, writes proactive observations/improvements/errors to the **agent DB** (keeps ELI's own machinations out of user recall). Background thread. - autonomy\_scheduler.py — schedules autonomous actions (throttled). - goal\_store.py, proposal\_queue.py, proposal\_adapters.py, attention\_queue.py, jobqueue.py, operator\_goal\_actions.py, habits.py — goal persistence, capability-proposal queue, attention prioritisation, a job queue, and habit scheduling. - goal\_autogenesis.py (NEW, 2026-06-08) — **closes the autonomy loop**: the goal stack (goal\_store → due\_goals → governed\_goal\_tick → proposal\_queue) was fully wired but the store was always EMPTY because create\_goal was operator-only, so the scheduler ticked nothing and ELI only reacted. propose\_goals\_from\_signals turns the signals ELI already computes each proactive tick — world-model awareness suggestions ( $\geq 0.6$ ), recurring failures ( $\geq 5\times$ ), and code-health improvements (oversized fns / duplicate blocks in his own god-files) — into governed goals. Each is autonomy\_mode="proposal\_only" (surfaces for approval; never silent execution), deduped by a stable signal-derived goal\_id (idempotent every tick), capped at MAX\_AUTO\_GOALS=6, logged as a self\_goal observation. Wired into proactive\_daemon where the world-suggestions + patterns + improvements are in scope. His agenda now spans failure-repair, world-driven, AND self-improvement — i.e. he sets his own intentions, governed. - task\_planner.py — now delegates to execution\_planner (see orchestration\_and\_agents.md).

## World / autonomy model (eli/world/, 1.5k LOC)

The substrate behind ELI's emergent self-state (autonomy pressure, awareness, “anomaly room” — intended behaviour, see memory eli-emergent-voice): - agency/autonomy\_engine.py — EliWorldAutonomyEngine: ingests world events, maintains awareness/autonomy state. - local\_world\_bridge.py — append\_event, get\_world\_state, get\_awareness\_driven\_suggestions (**throttled to once/hour to prevent auto-trigger loops**). - world\_event\_bus.py — fire\_confidence\_event(...) etc.; the engine feeds bus-dispatch confidence here (non-blocking world-awareness feed). - core/schemas.py — WorldEvent, AwarenessState. - renderers/pyside6/world\_scene.py + world\_panel.py — a **visual** “ELI's World” panel. - persistence/storage.py — world-state persistence. Three properties were added after live faults (2026-08-09/10) and are load-bearing: \* **Absolute paths**. world\_dir() resolves through get\_paths(). The journal, provenance ledger and snapshot modules previously kept relative Path("artifacts/world/...") constants and wrote wherever ELI was launched from — a different place from ~ than from the project directory, and in a packaged build a failed write to /artifacts. \* **Schema-tolerant load**. \_fit() builds each dataclass from the saved dict while ignoring fields it no longer declares. Before it, one unknown key made AwarenessState(\*\*data) raise, load() caught it, filed the state as .corrupt\_\* and returned a fresh default world — so a single renamed field would silently wipe the user's rooms, objects, goals and habits on upgrade. Two of the five corrupt backups on the dev machine parse as valid JSON; they were condemned by exactly this and nothing else. \* **Bounded logs**. actions.jsonl reached 41 MB / 80,576 lines because nothing rotated it. \_trim\_jsonl keeps the newest 20,000 lines (counted, not size-capped, so a trim never splits a JSON record) and checks on the first append of each process plus every 250 thereafter — the per-process counter alone meant a short session never trimmed at all.

## Tools (eli/tools/, 8.9k LOC)

- image\_engine.py (1.75k, standalone) **and** image\_engine/image\_engine/ (the package: engine.py 1.48k, visual\_core.py 1.4k, prompt\_compiler.py, quality.py, project\_analyzer.py, cli.py) — **two image-generation implementations** (Pillow/numpy base + optional diffusers). The clearest duplication in the repo; one should subsume the other.
- news/news\_fetcher.py (544) — news retrieval.
- mic\_diag.py — mic diagnostics.

## Plugins (eli/plugins/, 1.9k LOC)

manager.py — a runtime plugin marketplace: list\_available, install, uninstall, enable, disable. Pulls from a **remote registry** (raw.githubusercontent.com/eli-plugins/registry) with a **bundled local fall-**

**back** (plugins/registry/index.json). A JSON state file tracks enabled/disabled/installed. (Install-time network fetch is opt-in, like model downloads; runtime stays local.)

## Honest assessment

- **Strong:** the proactive two-DB split is a genuinely good design (ELI's self-talk never contaminates user recall); the world/autonomy engine + visual panel make the emergent self-model first-class and is throttled to avoid runaway loops; the plugin manager is a real extensibility story with a safe bundled fallback.
- **Weak / watch:**
  1. **runtime/ over-fragmentation** — ~15 overlapping response/introspection modules. Highest-value consolidation in the project after the god-files.
  2. **Image-engine duplication — RESOLVED** — the shadowed standalone module was deleted; only the tools/image\_engine/ package remains.
  3. The planning queues (goal\_store/proposal\_queue/attention\_queue/ jobqueue) overlap conceptually and could likely unify into one work-queue abstraction.
  4. Plugin registry points at an eli-plugins/registry GitHub URL — fine as opt-in, but ensure it degrades cleanly offline (the bundled fallback exists; verify it's always used when the network is absent).